

Learning

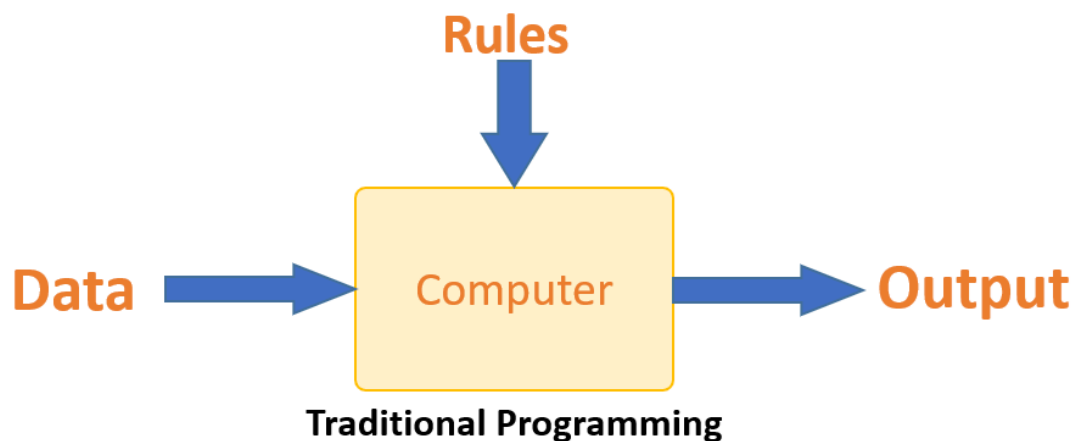
According to **Herbert Simon**, learning denotes changes in a system that enable a system to do the same task more efficiently the next time.

The main purpose of machine learning is to study and design the algorithms that can be used to produce the predicates from the given dataset.

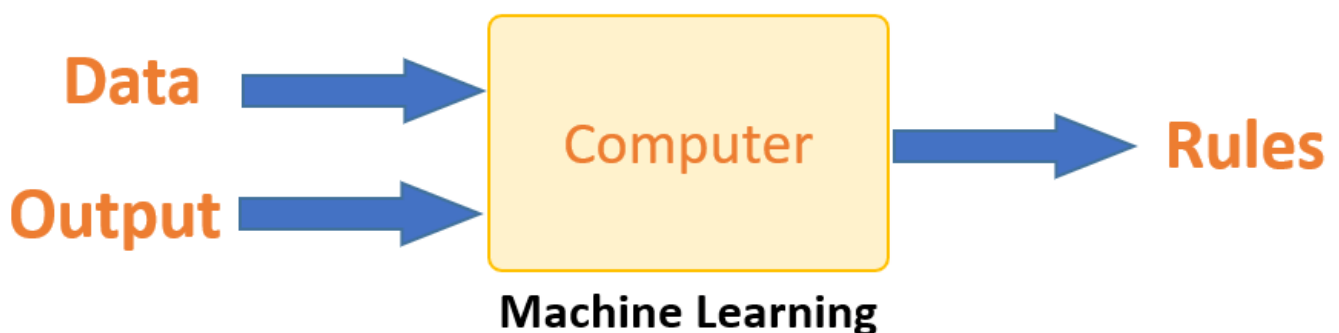
Besides these, the machine learning includes the agents' percepts for acting as well as to improve their future performance.

Machine Learning vs. Traditional Programming

Traditional programming differs significantly from machine learning. In traditional programming, a programmer codes all the rules in consultation with an expert in the industry for which software is being developed. Each rule is based on a logical foundation; the machine will execute an output following the logical statement. When the system grows complex, more rules need to be written. It can quickly become unsustainable to maintain.



Machine learning is supposed to overcome this issue. The machine learns how the input and output data are correlated and it writes a rule. The programmers do not need to write new rules each time there is new data. The algorithms adapt in response to new data and experiences to improve efficacy over time.

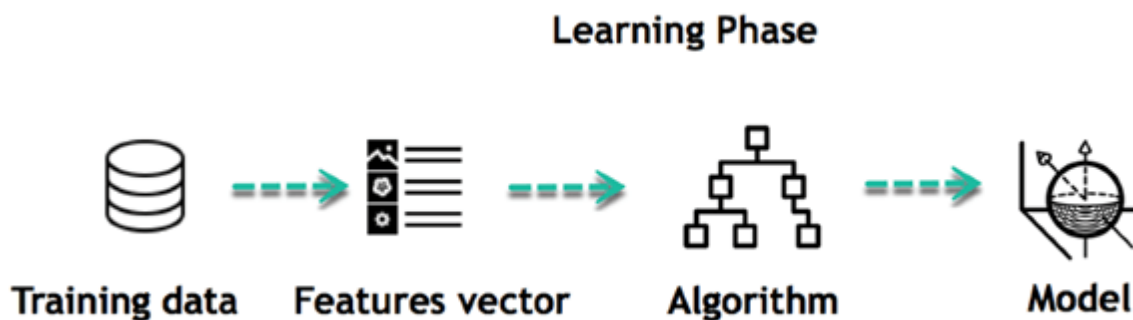


How does Machine learning work?

Machine learning is the brain where all the learning takes place. The way the machine learns is similar to the human being. Humans learn from experience. The more we know, the more easily we can predict. By analogy, when we face an unknown situation, the likelihood of success is lower than the known situation. Machines are trained the same. To make an accurate prediction, the machine sees an example. When we give the machine a similar example, it can figure out the outcome. However, like a human, if its feed a previously unseen example, the machine has difficulties to predict.

The core objective of machine learning is the **learning** and **inference**. First of all, the machine learns through the discovery of patterns. This discovery is made thanks to the **data**. One crucial part of the data scientist is to choose carefully which data to provide to the machine. The list of attributes used to solve a problem is called a **feature vector**. You can think of a feature vector as a subset of data that is used to tackle a problem.

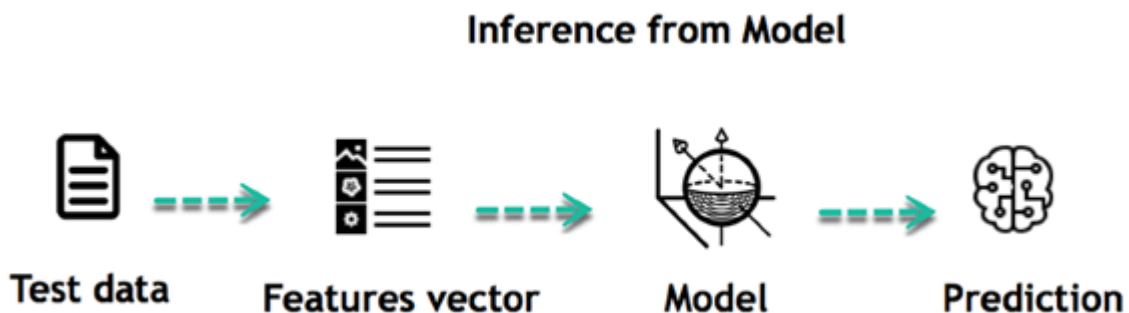
The machine uses some fancy algorithms to simplify the reality and transform this discovery into a **model**. Therefore, the learning stage is used to describe the data and summarize it into a model.



For instance, the machine is trying to understand the relationship between the wage of an individual and the likelihood to go to a fancy restaurant. It turns out the machine finds a positive relationship between wage and going to a high-end restaurant: This is the model

Inferring

When the model is built, it is possible to test how powerful it is on never-seen-before data. The new data are transformed into a features vector, go through the model and give a prediction. This is all the beautiful part of machine learning. There is no need to update the rules or train again the model. You can use the model previously trained to make inference on new data.

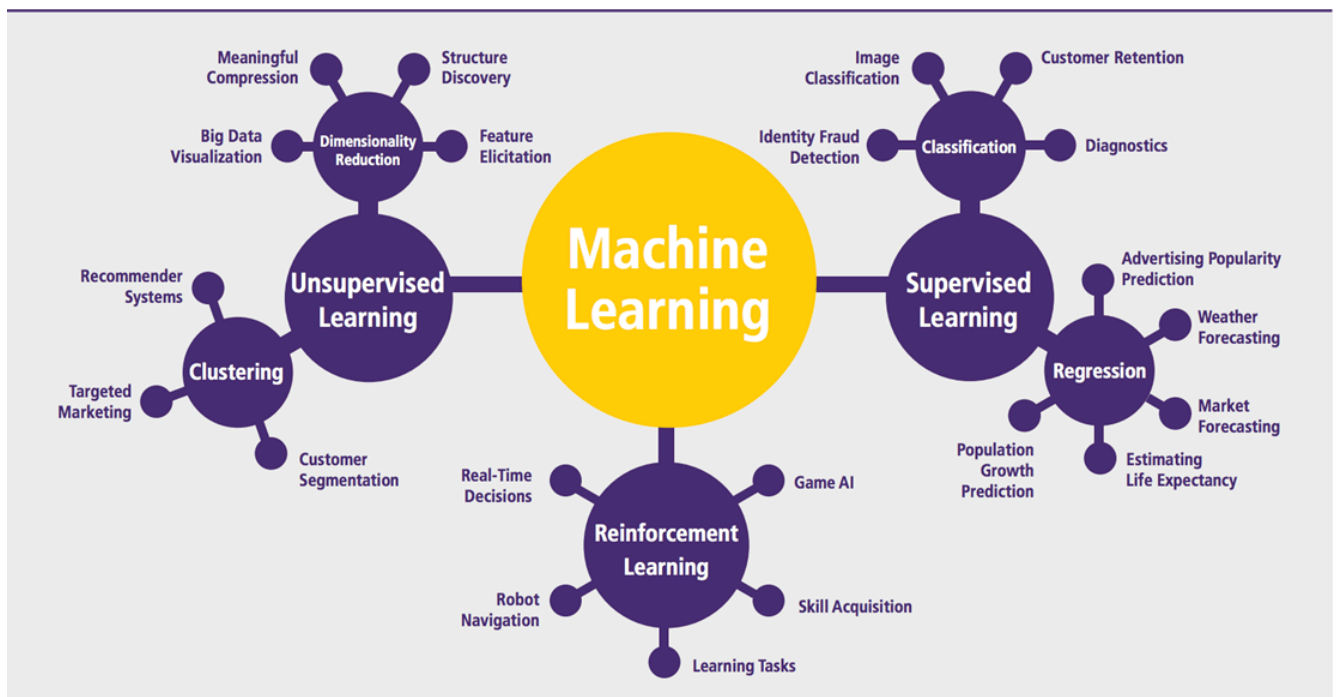


The life of Machine Learning programs is straightforward and can be summarized in the following points:

1. Define a question
2. Collect data
3. Visualize data
4. Train algorithm
5. Test the Algorithm
6. Collect feedback
7. Refine the algorithm
8. Loop 4-7 until the results are satisfying
9. Use the model to make a prediction

Once the algorithm gets good at drawing the right conclusions, it applies that knowledge to new sets of data.

Machine learning Algorithms and where they are used?



Machine learning can be grouped into two broad learning tasks: Supervised and Unsupervised. There are many other algorithms

Learning Agent

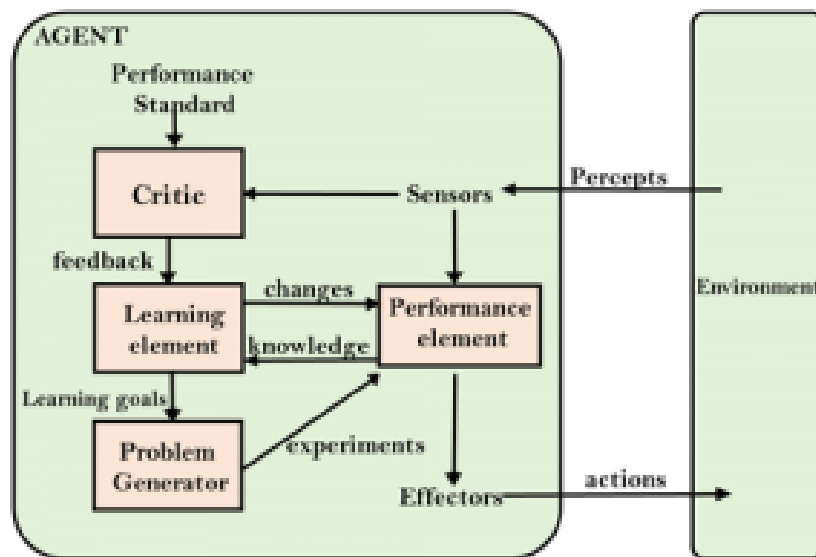
A learning agent in AI is the type of agent which can learn from its past experiences or it has learning capabilities.

It starts to act with basic knowledge and then able to act and adapt automatically through learning.

A learning agent has mainly four conceptual components, which are:

1. **Learning element** :It is responsible for making improvements by learning from the environment

2. **Critic:** Learning element takes feedback from critic which describes how well the agent is doing with respect to a fixed performance standard.
3. **Performance element:** It is responsible for selecting external action
4. **Problem Generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.



The following tasks must be learned by an agent.

- To predict or decide the result state for an action.
- To know the values for each state (understand which state has high or low value).
- To keep record of relevant percepts.

Why do we require machine learning?

- Machine learning plays an important role in improving and understanding the efficiency of human learning.
- Machine learning is used to discover new things not known to many human beings.

Various forms of learnings are explained below:

1. Rote learning

- Rote learning is possible on the basis of memorization.
- This technique mainly focuses on memorization by avoiding the inner complexities. So, it becomes possible for the learner to recall the stored knowledge.

For example: When a learner learns a poem or song by reciting or repeating it, without knowing the actual meaning of the poem or song.

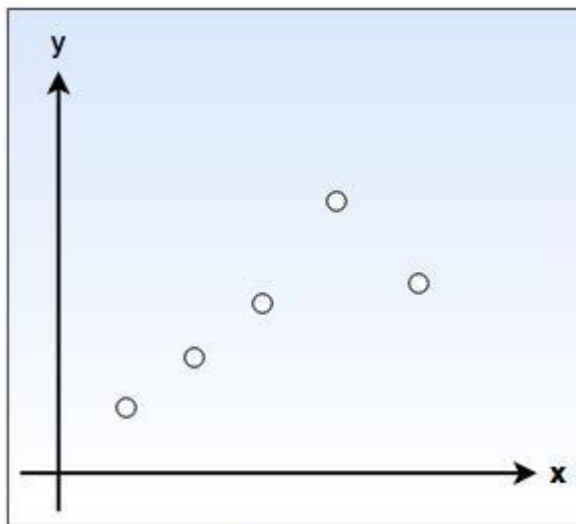
2. Induction learning (Learning by example).

- Induction learning is carried out on the basis of supervised learning.
- In this learning process, a general rule is induced by the system from a set of observed instance.
- **For Example:**

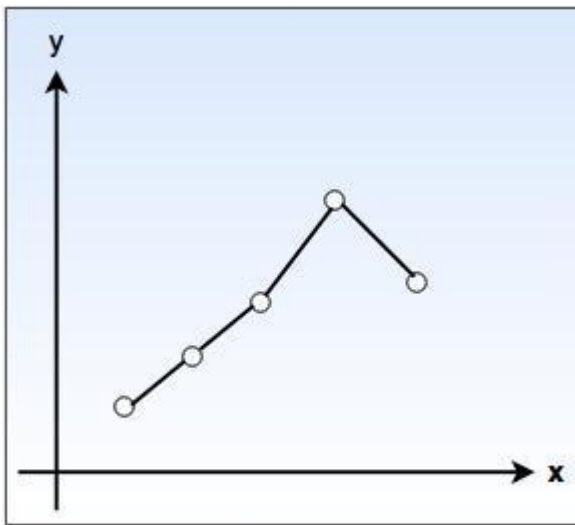
Consider that ' f ' is the target function and example is a pair $(x, f(x))$, where ' x ' is input and $f(x)$ is the output function applied to ' x '.

Given problem: Find hypothesis h such as $h \approx f$

- So, in the following fig-a, points (x,y) are given in plane so that $y = f(x)$, and the task is to find a function $h(x)$ that fits the point well.

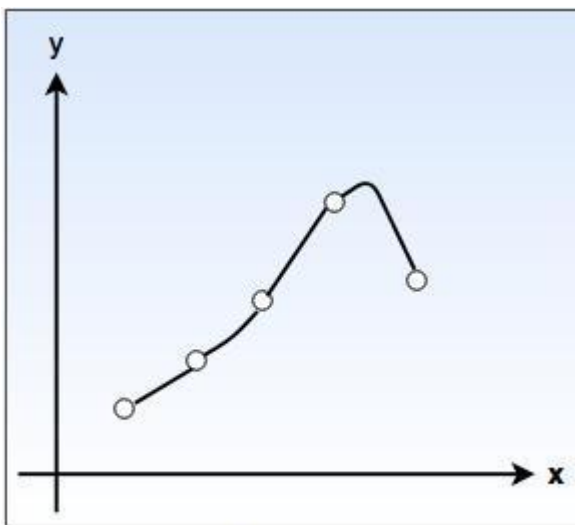


Fig(a)



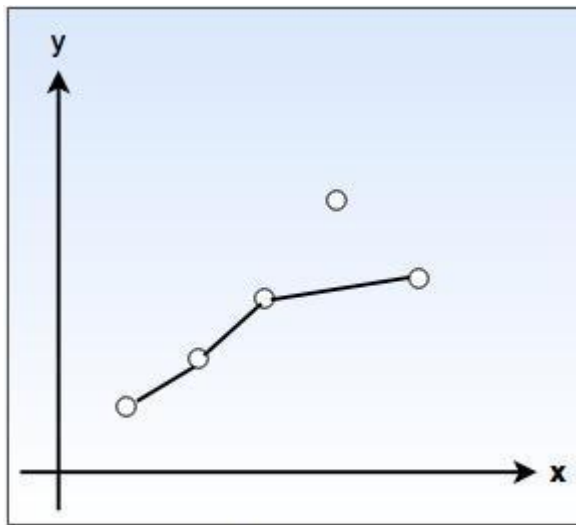
Fig(b)

- In fig-b, a piecewise-linear 'h' function is given, while the fig-c shows more complicated 'h' function.



Fig(c)

- Both the functions agree with the example points, but differ with the values of 'y' assigned to other x inputs.



Fig(d)

- As shown in fig.(d), we have a function that apparently ignores one of the example points, but fits others with a simple function.

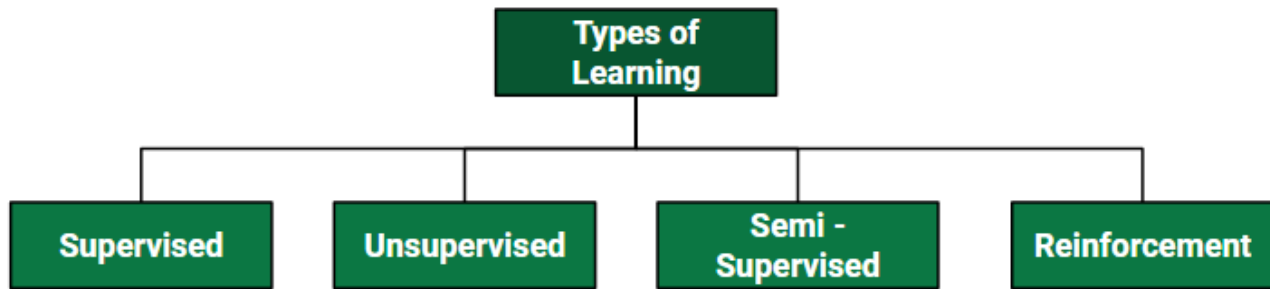
Explanation based learning

- Explanation-based learning (EBL) deals with an idea of single-example learning.
- This type of learning usually requires a substantial number of training instances but there are two difficulties in this:
 - I. it is difficult to have such a number of training instances
 - ii. Sometimes, it may help us to learn certain things effectively, specially when we have enough knowledge.

Hence, it is clear that instance-based learning is more data-intensive, data-driven while EBL is more knowledge-intensive, knowledge-driven.

- Initially, an EBL system accepts a training example.
- On the basis of the given goal concept, an operability criteria and domain theory, it "generalizes" the training example to describe the goal concept and to satisfy the operability criteria (which are usually a set of rules that describe relationships between objects and actions in a domain).
- Thus, several applications are possible for the knowledge acquisition and engineering aspects.

Supervised, Unsupervised, semisupervised and reinforcement Learning



Supervised Learning :

Supervised learning is when the model is getting trained on a labelled dataset. **Labelled** dataset is one which have both input and output parameters. In this type of learning both training and validation datasets are labelled as shown in the figures below.

User ID	Gender	Age	Salary	Purchased	Temperature	Pressure	Relative Humidity	Wind Direction	Wind Speed
15624510	Male	19	19000	0	10.69261758	986.882019	54.19337313	195.7150879	3.278597116
15810944	Male	35	20000	1	13.59184184	987.8729248	48.0648859	189.2951202	2.909167767
15668575	Female	26	43000	0	17.70494885	988.1119385	39.11965597	192.9273834	2.973036289
15603246	Female	27	57000	0	20.95430404	987.8500366	30.66273218	202.0752869	2.965289593
15804002	Male	19	76000	1	22.9278274	987.2833862	26.06723423	210.6589203	2.798230886
15728773	Male	27	58000	1	24.04233986	986.2907104	23.46918024	221.1188507	2.627005816
15598044	Female	27	84000	0	24.41475295	985.2338867	22.25082295	233.7911987	2.448749781
15694829	Female	32	150000	1	23.93361956	984.8914795	22.35178837	244.3504333	2.454271793
15600575	Male	25	33000	1	22.68800023	984.8461304	23.7538641	253.0864716	2.418341875
15727311	Female	35	65000	0	20.56425726	984.8380737	27.07867944	264.5071106	2.318677425
15570769	Female	26	80000	1	17.76400389	985.4262085	33.54900114	280.7827454	2.343950987
15606274	Female	26	52000	0	11.25680746	988.9386597	53.74139903	68.15406036	1.650191426
15746139	Male	20	86000	1	14.37810685	989.6819458	40.70884681	72.62069702	1.553469896
15704987	Male	32	18000	0	18.45114201	990.2960205	30.85038484	71.70604706	1.005017161
15628972	Male	18	82000	0	22.54895853	989.9562988	22.81738811	44.66042709	0.264133632
15697686	Male	29	80000	0	24.23155922	988.796875	19.74790765	318.3214111	0.329656571
15733883	Male	47	25000	1					

Figure A: CLASSIFICATION

Figure B: REGRESSION

Both the above figures have labelled data set –

- Figure A:** It is a dataset of a shopping store which is useful in predicting whether a customer will purchase a particular product under consideration or not based on his/ her gender, age and salary.
Input : Gender, Age, Salary
Output : Purchased i.e. 0 or 1 ; 1 means yes the customer will purchase and 0 means that customer won't purchase it.
- Figure B:** It is a Meteorological dataset which serves the purpose of predicting wind speed based on different parameters.

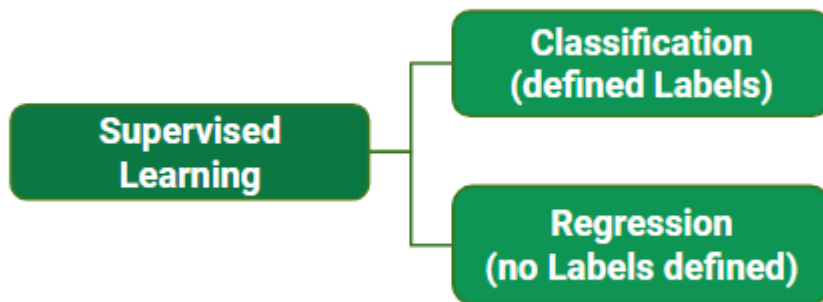
Input : Dew Point, Temperature, Pressure, Relative Humidity, Wind Direction

Output : Wind Speed

Training the system:

While training the model, data is usually split in the ratio of 80:20 i.e. 80% as training data and rest as testing data. In training data, we feed input as well as output for 80% data. The model learns from training data only. We use different machine learning algorithms to build our model. By learning, it means that the model will build some logic of its own.

Once the model is ready then it is good to be tested. At the time of testing, input is fed from remaining 20% data which the model has never seen before, the model will predict some value and we will compare it with actual output and calculate the accuracy.



Types of Supervised Learning:

1. **Classification :** It is a Supervised Learning task where output is having defined labels(discrete value). For example in above Figure A, Output – Purchased has defined labels i.e. 0 or 1 ; 1 means the customer will purchase and 0 means that customer won't purchase. The goal here is to predict discrete values belonging to a particular class and evaluate on the basis of accuracy. It can be either binary or multi class classification. In **binary** classification, model predicts either 0 or 1 ; yes or no but in case of **multi class** classification, model predicts more than one class.

Example: Gmail classifies mails in more than one classes like social, promotions, updates, forum.

2. **Regression :** It is a Supervised Learning task where output is having continuous value. Example in above Figure B, Output – Wind Speed is not having any discrete value but is continuous in the particular range. The goal here is to predict a value as much closer to actual

output value as our model can and then evaluation is done by calculating error value. The smaller the error the greater the accuracy of our regression model.

Example of Supervised Learning Algorithms:

- Linear Regression
- Nearest Neighbor
- Guassian Naive Bayes
- Decision Trees
- Support Vector Machine (SVM)
- Random Forest

Unsupervised Learning :

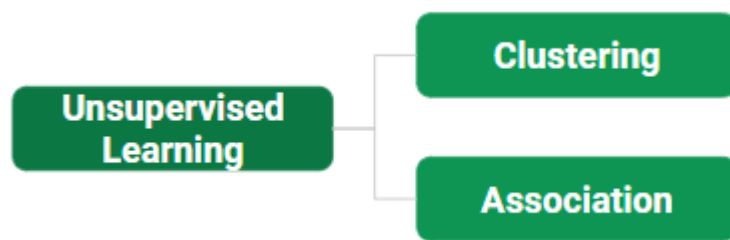
It's a type of learning where we don't give target to our model while training i.e. training model has only input parameter values. The model by itself has to find which way it can learn. Data-set in Figure A is mall data that contains information of its clients that subscribe to them. Once subscribed they are provided a membership card and so the mall has complete information about customer and his/her every purchase. Now using this data and unsupervised learning techniques, mall can easily group clients based on the parameters we are feeding in.

CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40
6	Female	22	17	76
7	Female	35	18	6
8	Female	23	18	94
9	Male	64	19	3
10	Female	30	19	72
11	Male	67	19	14
12	Female	35	19	99
13	Female	58	20	15
14	Female	24	20	77
15	Male	37	20	13
16	Male	22	20	79
17	Female	35	21	35

Figure A

Training data we are feeding is –

- **Unstructured data:** May contain noisy(meaningless) data, missing values or unknown data
- **Unlabeled data :** Data only contains value for input parameters, there is no targeted value(output). It is easy to collect as compared to labelled one in Supervised approach.



Types of Unsupervised Learning :-

- **Clustering:** Broadly this technique is applied to group data based on different patterns, our machine model finds. For example in above figure we are not given output parameter value, so this technique will be used to group clients based on the input parameters provided by our data.
- **Association:** This technique is a rule based ML technique which finds out some very useful relations between parameters of a large data set. For e.g. shopping stores use algorithms based on this technique to find out relationship between sale of one product w.r.t to others sale based on customer behavior. Once trained well, such models can be used to increase their sales by planning different offers.

Some algorithms:

- 🎬 K-Means Clustering
- 🎬 DBSCAN – Density-Based Spatial Clustering of Applications with Noise
- 🎬 BIRCH – Balanced Iterative Reducing and Clustering using Hierarchies
- 🎬 Hierarchical Clustering

Semi-supervised Learning:

As the name suggests, its working lies between Supervised and Unsupervised techniques. We use these techniques when we are dealing with a data which is a little bit labelled and rest large portion of it is unlabeled. We can use unsupervised technique to predict labels and then feed these labels to supervised techniques. This technique is mostly applicable in case of image data-sets where usually all images are not labelled.



Reinforcement Learning:

In this technique, model keeps on increasing its performance using a Reward Feedback to learn the behavior or pattern. These algorithms are specific to a particular problem e.g. Google Self Driving car, AlphaGo where a bot competes with human and even itself to getting better and better performer of Go Game. Each time we feed in data, they learn and add the data to its knowledge that is training data. So, more it learns the better it get trained and hence experienced.

- Agents observe input.
- Agent performs an action by making some decisions.
- After its performance, agent receives reward and accordingly reinforce and the model stores in state-action pair of information.

Some algorithms:

- Temporal Difference (TD)
- Q-Learning
- Deep Adversarial Networks

Decision Tree

It's time to begin the journey

Decision tree builds classification or regression models in the form of a tree structure. It breaks down a dataset into smaller subsets with increase in depth of tree. The final result is a tree with **decision nodes** and **leaf nodes**. A decision node (e.g., Outlook) has two or more branches (e.g., Sunny, Overcast and Rainy). Leaf node (e.g., Play) represents a classification or decision. The topmost decision node in a tree which corresponds to the best predictor is called **root node**. Decision trees can handle both categorical and numerical data.

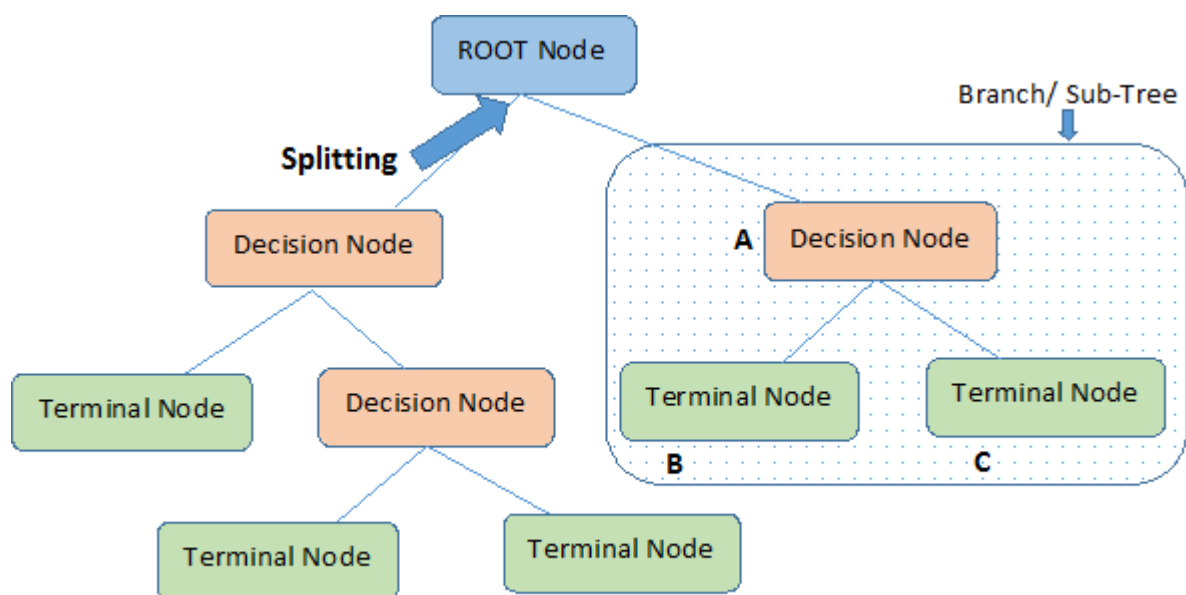


Types of decision trees

1. Categorical Variable Decision Tree: Decision Tree which has categorical target variable then it called as categorical variable decision tree.
2. Continuous Variable Decision Tree: Decision Tree which has continuous target variable then it is called as Continuous Variable Decision Tree.

Important Terminology related to Decision Trees

Let's look at the basic terminologies used with Decision trees:



Note:- A is parent node of B and C.

1. **Root Node:** It represents entire population or sample and this further gets divided into two or more homogeneous sets.
2. **Splitting:** It is a process of dividing a node into two or more sub-nodes.
3. **Decision Node:** When a sub-node splits into further sub-nodes, then it is called decision node.
4. **Leaf/ Terminal Node:** Nodes with no children (no further split) is called Leaf or Terminal node.
5. **Pruning:** When we reduce the size of decision trees by removing nodes (opposite of Splitting), the process is called pruning.
6. **Branch / Sub-Tree:** A sub section of decision tree is called branch or sub-tree.
7. **Parent and Child Node:** A node, which is divided into sub-nodes is called parent node of sub-nodes where as sub-nodes are the child of parent node.

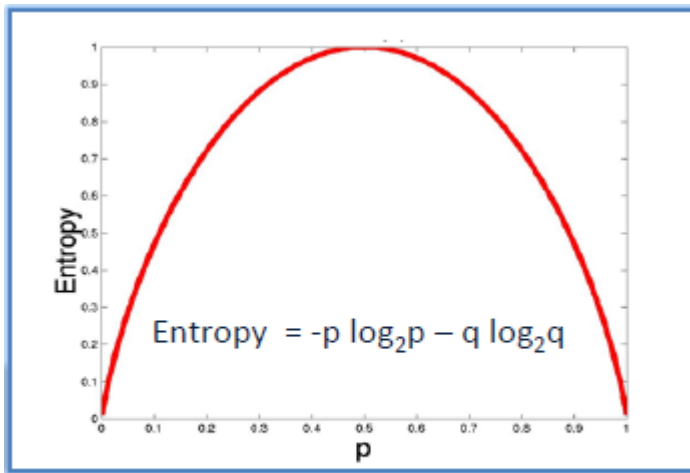
Algorithm used in decision trees:

1. ID3
2. Gini Index
3. Chi-Square
4. Reduction in Variance

ID3

The core algorithm for building decision trees is called **ID3**. Developed by J. R. Quinlan, this algorithm employs a top-down, greedy search through the space of possible branches with no backtracking. ID3 uses *Entropy* and *Information Gain* to construct a decision tree.

Entropy : A decision tree is built top-down from a root node and involves partitioning the data into subsets that contain instances with similar values (homogeneous). ID3 algorithm uses entropy to calculate the homogeneity of a sample. If the sample is completely homogeneous the entropy is zero and if the sample is equally divided then it has entropy of one.



$$\text{Entropy} = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$$

To build a decision tree, we need to calculate two types of entropy using frequency tables as follows:

a) Entropy using the frequency table of one attribute:

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5



$$\begin{aligned} \text{Entropy}(\text{PlayGolf}) &= \text{Entropy}(5,9) \\ &= \text{Entropy}(0.36, 0.64) \\ &= -(0.36 \log_2 0.36) - (0.64 \log_2 0.64) \\ &= 0.94 \end{aligned}$$

b) Entropy using the frequency table of two attributes:

$$E(T, X) = \sum_{c \in X} P(c)E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14



$$\begin{aligned}
 E(\text{PlayGolf}, \text{Outlook}) &= P(\text{Sunny}) * E(3,2) + P(\text{Overcast}) * E(4,0) + P(\text{Rainy}) * E(2,3) \\
 &= (5/14) * 0.971 + (4/14) * 0.0 + (5/14) * 0.971 \\
 &= 0.693
 \end{aligned}$$

Information Gain: The information gain is based on the decrease in entropy after a data-set is split on an attribute. Constructing a decision tree is all about finding attribute that returns the highest information gain (i.e., the most homogeneous branches).

Step 1: Calculate entropy of the target.

$$\begin{aligned}
 \text{Entropy}(\text{PlayGolf}) &= \text{Entropy}(5,9) \\
 &= \text{Entropy}(0.36, 0.64) \\
 &= -(0.36 \log_2 0.36) - (0.64 \log_2 0.64) \\
 &= 0.94
 \end{aligned}$$

Step 2: The dataset is then split on the different attributes. The entropy for each branch is calculated. Then it is added proportionally, to get total entropy for the split. The resulting entropy is subtracted from the entropy before the split. The result is the Information Gain, or decrease in entropy.

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
Gain = 0.029			

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
Gain = 0.152			

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
Gain = 0.048			

$$Gain(T, X) = Entropy(T) - Entropy(T, X)$$

$$G(\text{PlayGolf}, \text{Outlook}) = E(\text{PlayGolf}) - E(\text{PlayGolf}, \text{Outlook})$$

$$= 0.940 - 0.693 = 0.247$$

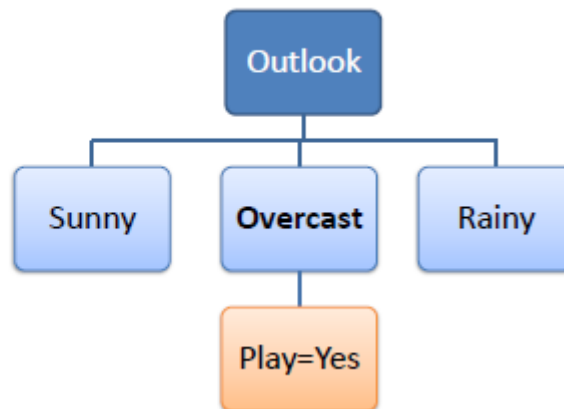
Step 3: Choose attribute with the largest information gain as the decision node, divide the dataset by its branches and repeat the same process on every branch.

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

		Outlook	Temp	Humidity	Windy	Play Golf
Outlook	Sunny	Sunny	Mild	High	FALSE	Yes
		Sunny	Cool	Normal	FALSE	Yes
		Sunny	Cool	Normal	TRUE	No
		Sunny	Mild	Normal	FALSE	Yes
		Sunny	Mild	High	TRUE	No
	Overcast	Overcast	Hot	High	FALSE	Yes
		Overcast	Cool	Normal	TRUE	Yes
		Overcast	Mild	High	TRUE	Yes
		Overcast	Hot	Normal	FALSE	Yes
	Rainy	Rainy	Hot	High	FALSE	No
		Rainy	Hot	High	TRUE	No
		Rainy	Mild	High	FALSE	No
		Rainy	Cool	Normal	FALSE	Yes
		Rainy	Mild	Normal	TRUE	Yes

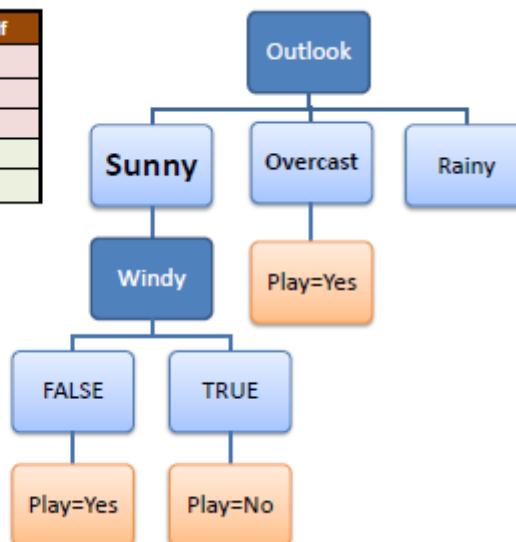
Step 4a: A branch with entropy of 0 is a leaf node.

Temp	Humidity	Windy	Play Golf
Hot	High	FALSE	Yes
Cool	Normal	TRUE	Yes
Mild	High	TRUE	Yes
Hot	Normal	FALSE	Yes



Step 4b: A branch with entropy more than 0 needs further splitting.

Temp	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



Step 5: The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified.

Example

Imagine you do a newspaper round to help you get through these lean times. On your round, you encounter a number of dogs that either bark or (try to) bite. The dogs are described by the following binary features: Heavy, Smelly, Big and Growling. Consider the following set of examples:

Heavy	Smelly	Big	Growling	Bites
No	No	No	No	No
No	No	Yes	No	No
Yes	Yes	No	Yes	No
Yes	No	No	Yes	Yes
No	Yes	Yes	No	Yes
No	No	Yes	Yes	Yes
No	No	No	Yes	Yes
Yes	Yes	No	No	Yes

question 1: What is the entropy of the target value Bites in the data?

$$H(\text{Bites}) = -\frac{5}{8} \log_2 \frac{5}{8} - \frac{3}{8} \log_2 \frac{3}{8} \approx 0.9544$$

question 2: Which attribute would the ID3 algorithm choose to use for the root of the tree (without pruning)?

Growling

question 3: What is the information gain of the attribute you chose in the previous question?

approximately 0.0487

question 4: Draw the full decision tree that would be learned for this data using ID3 without pruning.

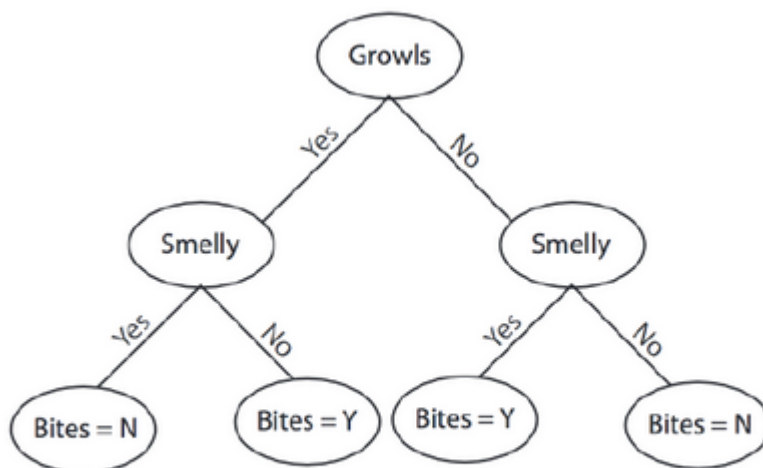


Figure 1: Decision tree

question 5: Suppose three new dogs appear in your round as listed in the table below. Classify them using the decision tree from the previous question.

Dog	Heavy	Smelly	Big	Growling	Bites
Buster	Yes	Yes	Yes	Yes	No
Pluto	No	Yes	No	Yes	No
Zeus	Yes	Yes	No	No	Yes

Gini Index

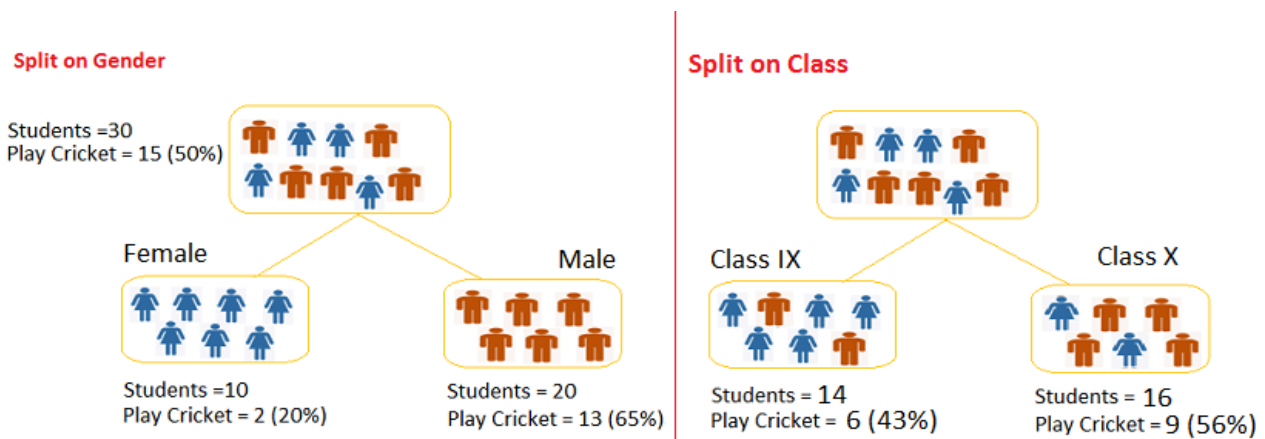
Gini index says, if we select two items from a population at random then they must be of same class and probability for this is 1 if population is pure.

1. It works with categorical target variable “Success” or “Failure”.
2. It performs only Binary splits
3. Higher the value of Gini higher the homogeneity.
4. CART (Classification and Regression Tree) uses Gini method to create binary splits.

Steps to Calculate Gini for a split

1. Calculate Gini for sub-nodes, using formula sum of square of probability for success and failure (p^2+q^2).
2. Calculate Gini for split using weighted Gini score of each node of that split

Example:—Referring to example where we want to segregate the students based on target variable (playing cricket or not). In the snapshot below, we split the population using two input variables Gender and Class. Now, I want to identify which split is producing more homogeneous sub-nodes using Gini index.



Split on Gender:

1. Gini for sub-node Female = $(0.2) \times (0.2) + (0.8) \times (0.8) = 0.68$
2. Gini for sub-node Male = $(0.65) \times (0.65) + (0.35) \times (0.35) = 0.55$
3. Weighted Gini for Split Gender = $(10/30) \times 0.68 + (20/30) \times 0.55 = \mathbf{0.59}$

Similar for Split on Class:

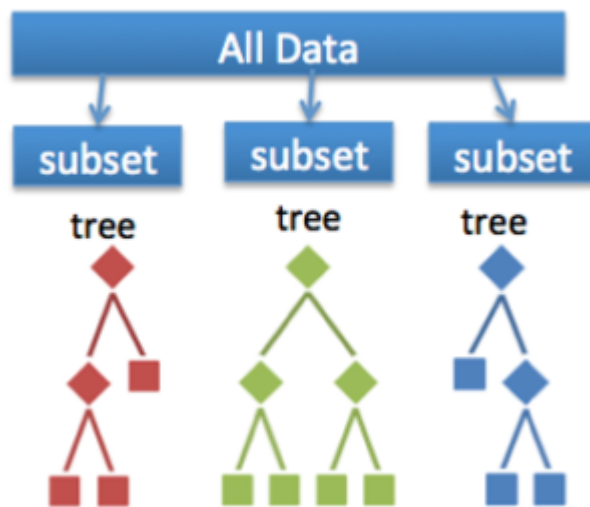
1. Gini for sub-node Class IX = $(0.43) \times (0.43) + (0.57) \times (0.57) = 0.51$
2. Gini for sub-node Class X = $(0.56) \times (0.56) + (0.44) \times (0.44) = 0.51$
3. Weighted Gini for Split Class = $(14/30) \times 0.51 + (16/30) \times 0.51 = \mathbf{0.51}$

Above, you can see that Gini score for *Split on Gender* is higher than *Split on Class*, hence, the node split will take place on Gender.

Random Forest Algorithm

Random forest algorithm is a supervised classification and regression algorithm. As the name suggests, this algorithm randomly creates a forest with several trees.

Generally, the more trees in the forest the more robust the forest looks like. Similarly, in the random forest classifier, the higher the number of trees in the forest, greater is the accuracy of the results.



Random Forest – Random Forest In R – Edureka

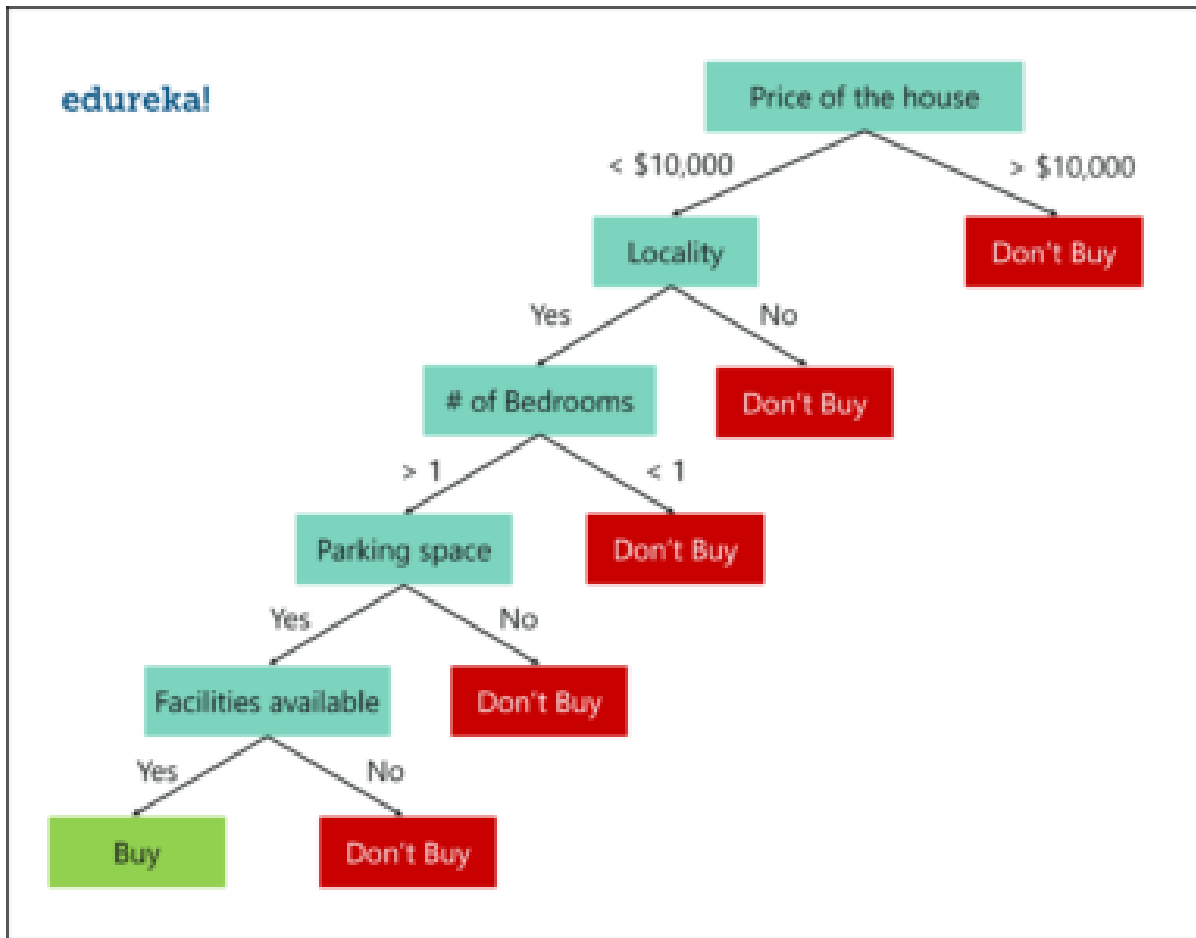
In simple words, Random forest builds multiple decision trees (called the forest) and glues them together to get a more accurate and stable prediction. The forest it builds is a collection of Decision Trees, trained with the bagging method.

Significance

Let's say that you're looking to buy a house, but you're unable to decide which one to buy. So, you consult a few agents and they give you a list of parameters that you should consider before buying a house. The list includes:

- Price of the house
- Locality
- Number of bedrooms
- Parking space
- Available facilities

These parameters are known as *predictor variables*, which are used to find the response variable. Here's a diagrammatic illustration of how you can represent the above problem statement using a decision tree.



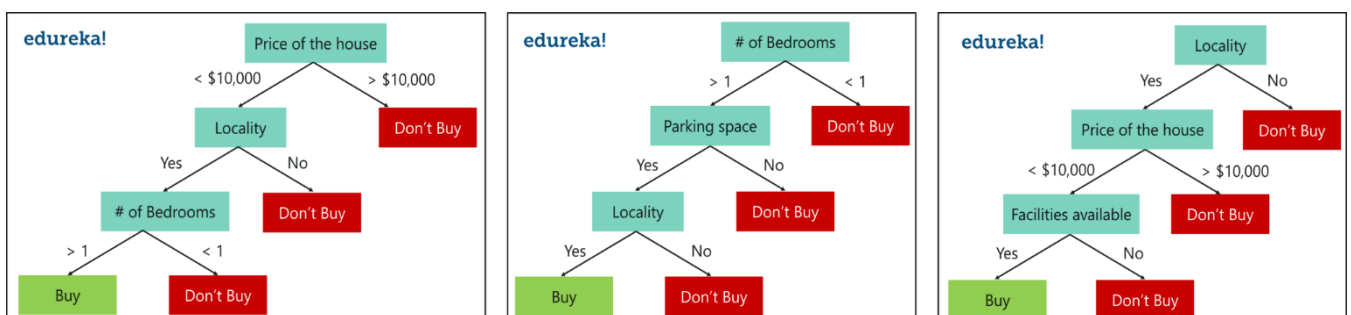
Decision Tree Example – Random Forest In R – Edureka

An important point to note here is that Decision trees are built on the entire data set, by making use of all the predictor variables.

Now let's see how Random Forest would solve the same problem.

Like mentioned earlier Random forest is an ensemble of decision trees, it randomly selects a set of parameters and creates a decision tree for each set of chosen parameters.

Take a look at the below figure.



Random Forest With 3 Decision Trees – Random Forest In R – Edureka

Here, 3 Decision Trees were created and each Decision Tree is taking only 3 parameters from the entire data set. Each decision tree predicts the outcome based on the respective predictor variables used in that tree and finally takes the average of the results from all the decision trees in the random forest.

In simple words, after creating multiple Decision trees using this method, each tree selects or votes the class (in this case the decision trees will choose whether or not a house is bought), and the class receiving the most votes by a simple majority is termed as the predicted class.

Thus, traditional decision trees are built on the entire data set using all the predictor variables, whereas Random Forests are used to create multiple decision trees, such that each decision tree is built only on a part of the data set.

Even though Decision trees are convenient and easily implemented, they lack accuracy. Decision trees work very effectively with the training data that was used to build them, but they're not flexible when it comes to classifying the new sample. Which means that the accuracy during testing phase is very low. This happens due to a process called Over-fitting.

Over-fitting occurs when a model studies the training data to such an extent that it negatively influences the performance of the model on new data.

This means that the disturbance in the training data is recorded and learned as concepts by the model. But the problem here is that these concepts do not apply to the testing data and negatively impact the model's ability to classify the new data, hence reducing the accuracy on the testing data.

How Does Random Forest Work?

To understand Random forest, consider the below sample data set. In this data set we have four predictor variables, namely:

- Weight
- Blood flow
- Blocked Arteries
- Chest Pain

Blood Flow	Blocked Arteries	Chest Pain	Weight	Heart Disease
Abnormal	No	No	130	No
Normal	Yes	Yes	195	Yes
Normal	No	Yes	218	No
Abnormal	Yes	Yes	180	Yes

Sample Data Set – Random Forest In R – Edureka

These variables are used to predict whether or not a person has heart disease. We're going to use this data set to create a Random Forest that predicts if a person has heart disease or not.

Creating A Random Forest

Step 1: Create a Bootstrapped Data Set

Bootstrapping is an estimation method used to make predictions on a data set by re-sampling it. To create a bootstrapped data set, we must randomly select samples from the original data set. A point to note here is that we can select the same sample more than once.

Blood Flow	Blocked Arteries	Chest Pain	Weight	Heart Disease
Normal	Yes	Yes	195	Yes
Abnormal	No	No	130	No
Abnormal	Yes	Yes	180	Yes
Abnormal	Yes	Yes	180	Yes

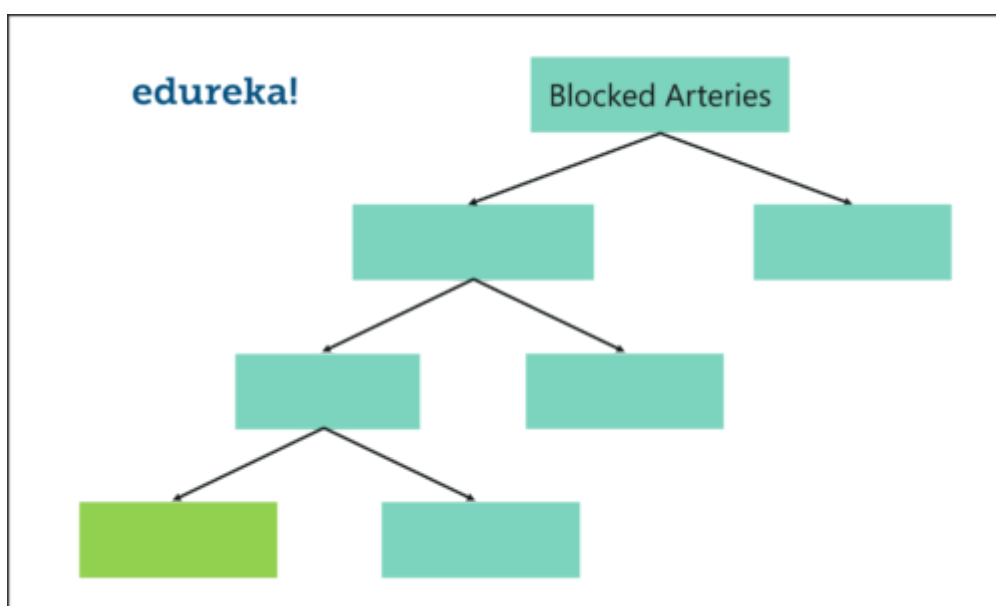
Bootstrapped Data Set – Random Forest In R – Edureka

In the above figure, selected samples are randomly selected from the original data set and created a bootstrapped data set. Well, in real-world problems you'll never get such a small data set, thus creating a bootstrapped data set is a little more complex.

Step 2: Creating Decision Trees

- Our next task is to build a Decision Tree by using the bootstrapped data set created in the previous step. Since we're making a Random Forest we will not consider the entire data set that we created, instead we'll only use a random subset of variables at each step.
- In this example, we're only going to consider two variables at each step. So, we begin at the root node, here we randomly select two variables as candidates for the root node.
- Let's say we selected Blood Flow and Blocked arteries. Out of these 2 variables, we must now select the variable that best separates the samples. For the sake of this example, let's say that Blocked Arteries is a more significant predictor and thus assign it as the root node.
- Our next step is to repeat the same process for each of the upcoming branch nodes. Here, we again select two variables at random as candidates for the branch node and then choose a variable that best separates the samples.

Just like this, we build the tree by only considering random subsets of variables at each step. By following the above process, our tree would look something like this:



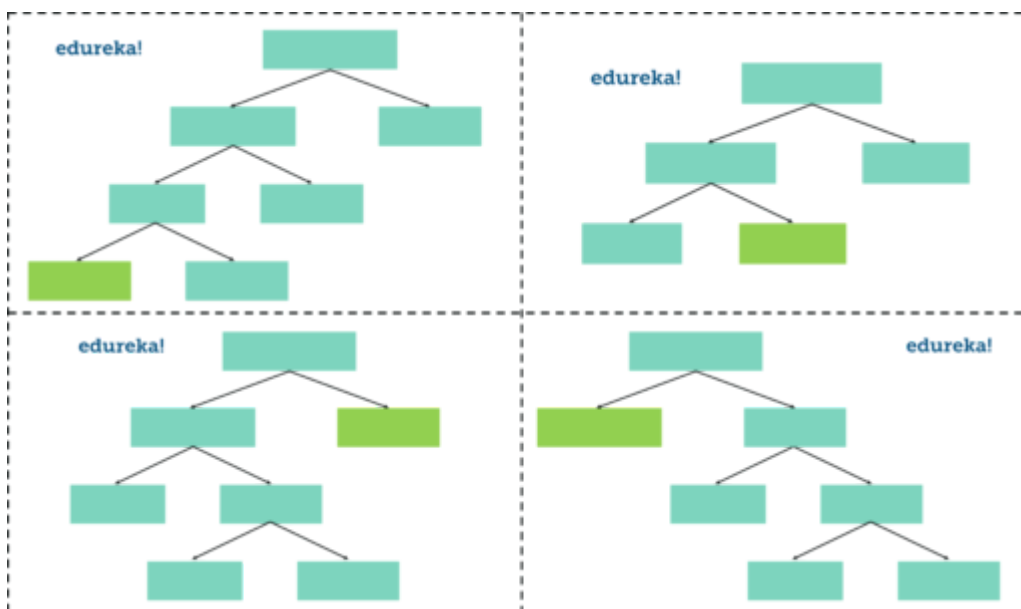
Random Forest Algorithm – Random Forest In R – Edureka

We just created our first Decision tree.

Step 3: Go back to Step 1 and Repeat

Like I mentioned earlier, Random Forest is a collection of Decision Trees. Each Decision Tree predicts the output class based on the respective predictor variables used in that tree. Finally, the outcome of all the Decision Trees in a Random Forest is recorded and the class with the majority votes is computed as the output class.

Thus, we must now create more decision trees by considering a subset of random predictor variables at each step. To do this, go back to step 1, create a new bootstrapped data set and then build a Decision Tree by considering only a subset of variables at each step. So, by following the above steps, our Random Forest would look something like this:



Random Forest – Random Forest In R – Edureka

This iteration is performed 100's of times, therefore creating multiple decision trees with each tree computing the output, by using a subset of randomly selected variables at each step.

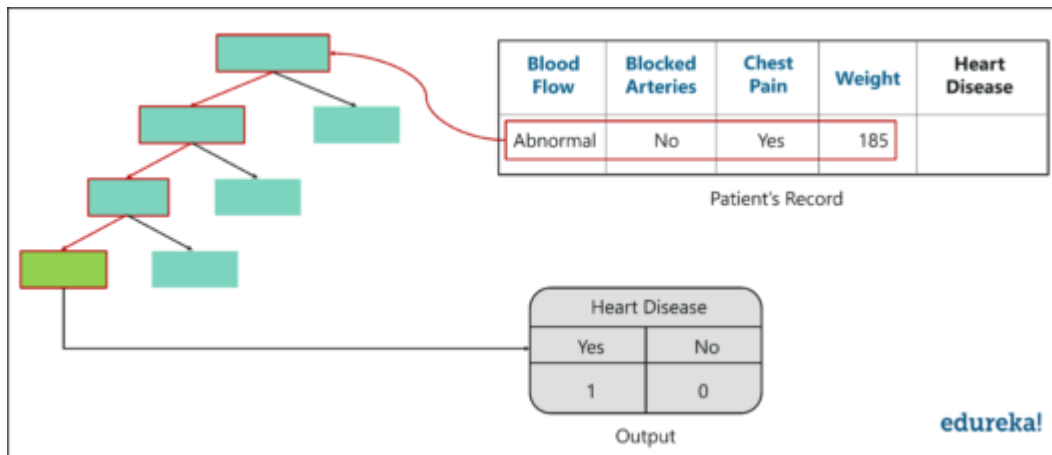
Having such a variety of Decision Trees in a Random Forest is what makes it more effective than an individual Decision Tree created using all the features and the whole data set.

Step 4: Predicting the outcome of a new data point

Now that we've created a random forest, let's see how it can be used to predict whether a new patient has heart disease or not.

The below diagram has the data about the new patient. All we have to do is run this data down the decision trees that we made.

The first tree shows that the patient has heart disease, so we keep a track of that in a table as shown in the figure.



Output – Random Forest In R – Edureka

Similarly, we run this data down the other decision trees and keep a track of the class predicted by each tree. After running the data down all the trees in the Random Forest, we check which class got the majority votes. In our case, the class 'Yes' received the most number of votes, hence it's clear that the new patient has heart disease.

To conclude, we bootstrapped the data and used the aggregate from all the trees to make a decision, this process is known as Bagging.

Step 5: Evaluate the Model

Our final step is to evaluate the Random Forest model. Earlier while we created the bootstrapped data set, we left out one entry/sample since we duplicated another sample. In a real-world problem, about 1/3rd of the original data set is not included in the bootstrapped data set.

The below figure shows the entry that didn't end up in the bootstrapped data set.

Blood Flow	Blocked Arteries	Chest Pain	Weight	Heart Disease
Normal	No	Yes	218	No

Out-Of-Bag Sample – Random Forest In R – Edureka

This sample data set that does not include in the bootstrapped data set is known as the Out-Of-Bag (OOB) data set.

The Out-Of-Bag data set is used to check the accuracy of the model, since the model wasn't created using this OOB data it will give us a good understanding of whether the model is effective or not.

In our case, the output class for the OOB data set is 'No'. So, in order for our Random Forest model to be accurate, if we run the OOB data down the Decision trees, we must get a majority of 'No' votes. This process is carried out for all the OOB samples, in our case we only had one OOB, however, in most problems, there are usually many more samples.

Therefore, eventually, we can measure the accuracy of a Random Forest by the proportion of OOB samples that are correctly classified.